

The 3-Tier Parallel Design Framework

In advanced PDC exam questions, you aren't just asked for one concept; you must design the entire architecture of a solution from scratch. The best way to format your answer for "Design a parallel algorithm for X" is to follow this strict 3-Tier hierarchy:

Tier 1: Choose the Task Routing Skeleton (How is work distributed?)

Your first decision is how the high-level tasks or data blocks are distributed among the processors.

- **Farm (Master-Worker):** Choose this if tasks are independent but take *unpredictable* amounts of time (e.g., Unsupervised KNN, Raytracing, parsing different sized files). It guarantees dynamic load balancing using a Queue.
- **Pipeline:** Choose this if data must pass through a strict sequence of different transformative stages (e.g., Read file -> Process text -> Write to Database, Video Encoding).
- **Divide & Conquer:** Choose this if the problem can be recursively halved and merged (e.g., Sorting arrays, Tree Traversals).
- **Wavefront:** Choose this if there is a 2D/3D grid with strict neighbor dependencies (e.g., Dynamic Programming, LCS, Heat Diffusion).

Tier 2: Choose the Data Processing Skeleton (What happens *inside* the task?)

Once a worker thread receives its block of work from Tier 1, what does it mathematically do with the data?

- **Map:** Choose this if the worker just needs to transform every element independently (e.g., squaring all numbers in its chunk, checking distances in KNN).
- **Reduce:** Choose this if the worker must aggregate a result into a single scalar value (e.g., finding the local Sum, or local Maximum). *Crucial requirement:* The mathematical operation **MUST** be associative.
- **Scan (Prefix Sum):** Choose this if the worker needs to compute running totals, cumulative states, or output an array based on partial accumulations across its chunk.

Tier 3: Choose the PRAM Model (How is shared memory accessed?)

Finally, you must justify the theoretical memory access rights your algorithm requires to function securely without race conditions.

- **EREW (Exclusive Read, Exclusive Write):** The safest, strictest model. Choose this if every thread operates on completely distinct, private indexes of an array and never reads a global variable simultaneously.
- **CREW (Concurrent Read, Exclusive Write):** The most common model. Choose this if all threads need to read a shared read-only variable (like a baseline threshold, or a shared search target in KNN), but they write their final answers to their own private output array indexes.
- **CRCW (Concurrent Read, Concurrent Write):** The most powerful theoretical model. Choose this **ONLY** if threads might simultaneously write to the exact same memory address (e.g., multiple threads finding

the maximum value and trying to overwrite a global `max_val` variable simultaneously). Be sure to specify how conflicts are resolved (Common, Arbitrary, Priority).

Real-World Exam Answers Using the 3-Tier Framework

If a professor asks you to design a system, writing your answer in this exact 3-step format guarantees you cover all the architectural requirements they are looking for.

Scenario 1: Unsupervised K-Nearest Neighbors (KNN)

Problem Statement: You have a massive dataset of 10 million points in 3D space. You must find the k closest neighbors for every single point. Because the density of the points varies wildly across the 3D space, some points find their neighbors instantly, while others require traversing deep into KD-Trees taking varying amounts of time. Design an architecture.

The 3-Tier Answer:

1. **Tier 1 (Task Skeleton):** I will use a **Farm (Master-Worker)** skeleton. Since searching dense vs. sparse areas takes unpredictable amounts of time, statically chunking the 10 million points (which a Map skeleton does) would lead to massive load imbalance. A Farm queue dynamically keeps all cores busy by feeding them the next point whenever they finish.
2. **Tier 2 (Data Processing):** Inside each worker, I will use a **Map** pattern to compute the Euclidean distance between the target point and all potential candidates locally and independently.
3. **Tier 3 (PRAM Model):** This operates correctly under a **CREW** model. All workers might implicitly need to *Concurrently Read* the shared massive KD-Tree data structure to find neighbors, but they will *Exclusively Write* their final lists of k neighbors to their own private array indices based on their assigned Point ID.

Architecture Code (Python):

```
import multiprocessing as mp

# Shared KD-Tree is read-only (CREW model)
global_kd_tree = build_kd_tree(all_points)

def worker_find_knn(task_queue, results):
    while True:
        target_point = task_queue.get()
        if target_point is None: break

        # Tier 2: MAP logic (independent computation)
        k_neighbors = global_kd_tree.search(target_point, k=5)

        # Exclusively writing to the results pipe
        results.put((target_point.id, k_neighbors))

# Tier 1: Farm Task Routing
workers = [mp.Process(target=worker_find_knn, args=(q, res)) for _ in range(8)]
```

Scenario 2: Parallel Document Word Counting

Problem Statement: You have 1,000 completely different text files (ranging from 1 KB scripts to 5 GB novels). You must output the total count of the word "Parallel" across all files combined.

The 3-Tier Answer:

1. **Tier 1 (Task Skeleton):** I will use a **Farm** skeleton. The files vary drastically in size (1KB to 5GB), meaning reading them takes drastically different times. The Master places file paths into a queue, and workers pop them dynamically.
2. **Tier 2 (Data Processing):** Once all workers finish counting the word "Parallel" inside their assigned files, we need a **Reduce** skeleton to safely sum all the individual file counts into one massive **Total_Count**.
3. **Tier 3 (PRAM Model):** Because we are reducing to a single **Total_Count** variable, if two workers finish their files at the exact same millisecond and try to add their local counts to the total simultaneously, it will cause a write conflict. Therefore, this theoretically requires a **CRCW (Concurrent Write)** model—specifically a *Common/Additive CRCW* where simultaneous writes are legally summed by the hardware. (In practice like OpenMP, we implement this CRCW requirement using an **atomic** or **reduction** clause).

Architecture Code (OpenMP translation of CRCW):

```
long long Total_Count = 0;

// OpenMP 'dynamic' schedule roughly emulates the Farm queue behavior (Tier 1)
// 'reduction' provides the safe Additive CRCW mechanism (Tier 2 & 3)
#pragma omp parallel for schedule(dynamic) reduction(+:Total_Count)
for(int i = 0; i < 1000; i++) {
    // Workers dynamically grab files, resolving Tier 1 load imbalance
    long local_count = count_word_in_file(files[i], "Parallel");
    Total_Count += local_count;
}
```

Scenario 3: Dynamic Programming Matrix (Longest Common Subsequence)

Problem Statement: Compare two DNA strings of length 100,000 by filling out a 2D scoring matrix where cell (i, j) depends mathematically on the answers previously computed at $(i-1, j)$ and $(i, j-1)$.

The 3-Tier Answer:

1. **Tier 1 (Task Skeleton):** I must use a **Wavefront** skeleton. The strict neighbor dependencies prohibit standard parallel Map loops. I will iterate diagonally across the matrix $k = 2 \dots (N+M)$. Each diagonal "wave" relies only on the previous waves.
2. **Tier 2 (Data Processing):** Within a specific diagonal k , all cells are completely independent of each other. Therefore, I will implicitly apply a **Map** skeleton over all valid (i, j) coordinate pairs *within that specific diagonal* to compute their scores simultaneously.
3. **Tier 3 (PRAM Model):** This operates safely under **CREW**. Multiple cells calculating on the current diagonal might need to *read* the exact same cell from the previous diagonal (Concurrent Read), but

each thread mathematically *writes* to a completely unique (i,j) cell address (Exclusive Write). No concurrent writes will ever happen.